

Frequency of the Most Frequent Element | Binary Search | Sliding Window
| META | Leetcode-1838

The frequency of an element is the number of times it occurs in an array.

You are given an integer array `nums` and an integer `k`. In one operation, you can choose an index of `nums` and increment the element at that index by 1.

Return the maximum possible frequency of an element after performing at most `k` operations.

Example 1:

Input: `nums = [1,2,4]`, `k = 5`

Output: 3

Explanation: Increment the first element three times and the second element two times to make `nums = [4,4,4]`.

4 has a frequency of 3.

Example 2:

Input: `nums = [1,4,8,13]`, `k = 5`

Output: 2

Explanation: There are multiple optimal solutions:

- Increment the first element three times to make `nums = [4,4,8,13]`.

4 has a frequency of 2.

- Increment the second element four times to make `nums = [1,8,8,13]`.

8 has a frequency of 2.

- Increment the third element five times to make `nums = [1,4,13,13]`.

13 has a frequency of 2.

Example 3:

Input: `nums = [3,9,6]`, `k = 2`

Output: 1

soln) most frequent element with highest frequency has to be exist in array.

ie. we cannot make other outside element as max freq element.

using binary search: if given array is not sorted then sort it.

as sorted use binary search

0 1 target-idx[2] count= (targetIdx - mid + 1) = 2 - 1 + 1 = 2

{ 1, 2, [4] } , K = 5

l | r
mid

windowInd= 4 * 2 = 8 (4+4) ie. count*num[target]

num[target]=4

Cur Sum = 2 + 4 = 6

8 - 6 = 2 ops. required to make freq 2

as we get target

we move r to mid-1

(if target is not found at mid then we move l to mid+1 as in binary search)

now,

0 1 target-idx[2] count= (targetIdx - mid + 1) = 2 - 0 + 1 = 3

{ 1, 2, [4] } , K = 5

l

r

mid

wi = 4 * 3 = 12 (4+4+4)

Cur Sum = 2 + 4 + 1 = 7

12 - 7 = 5 ops. required to make freq 3

maintain freq variable.

also check for targetIndex as 1,2 using binary search function

freq = binarysearch(target-ind,k,nums)

result = max(result,freq)

use prefix sum for cursum calculation

cursum=prefix[targetindex]-prefix[mid]+nums[mid]

har nums element ko target consider krke max freq ans find krenge

here count means elements from mid to target index

```

int bSearch(int target_idx, int k, vector<int>& nums,
            vector<long>& prefixSum) {

    int target = nums[target_idx];

    int l = 0;
    int r = target_idx;

    int best_idx = target_idx;

    while (l <= r) {
        int mid = l + (r - l) / 2;

        long count = (target_idx - mid + 1);
        long windowSum = count * target;

        long currSum = prefixSum[target_idx] - prefixSum[mid] + nums[mid];

        int ops = windowSum - currSum;

        if (ops > k) {
            l = mid + 1;
        } else {
            best_idx = mid;
            r = mid - 1;
        }
    }
    return target_idx - best_idx + 1;
}

```

```

int maxFrequency(vector<int>& nums, int k) {
    int n = nums.size();

    sort(begin(nums), end(nums));

    vector<long> prefixSum(n);
    prefixSum[0] = nums[0];
    for (int i = 1; i < n; i++) {
        prefixSum[i] = prefixSum[i - 1] + nums[i];
    }
}

```

```
int result = 0;
```

```
    for (int target_idx = 0; target_idx < n; target_idx++) {  
        result = max(result, bSearch(target_idx, k, nums, prefixSum));  
    }
```

```
    return result;  
}
```

